Design and Normalisation of a Healthcare Insurance Claims Database

CSCU9B3 Relational Database Assignment
Student number 3602723

Abstract – The following report describes a five table relational database for healthcare insurance claims that will be used for fraud analysis. It documents the current entities, keys, sample data, SQL query examples, and the normalisation properties that can be implemented. There is also SQL statements provided to show table definitions, one example insert for each entity, and CSV importing statements. There are also screenshots as evidence of query outputs

I. INTRODUCTION

The proposed database stores sample healthcare insurance claim records along with member, provider, diagnosis-code, and medicine information. It's purpose is to organise claims and related reference data so that an analyst can filter, join, aggregate, and update records as part of a fraud analysis workflow. The database stores claim attributes such as billed and paid amounts and an illustrative fraud score.

The implementation consists of five tables and five CSV files. Each CSV contains ten-ish data rows. This report explains the design and examples

II. DATABASE DESIGN

A. Entities and keys

The design contains members (member profile attributes) , providers (provider, name and status), ICD 10 ailment description (code and description), claims_data (claim facts and references), and ailment_medicine (medicine details linked to a diagnosis code). The primary keys are member_id, provider_id, icd_10_code, claim_id, and drug_id, respectively. The claims table references members, providers, and diagnosis codes. The medicine table references a diagnosis code.

In this implementation, a claims row requires one member, one provider, and one diagnosis code because those foreign-key columns are declared NOT NULL. A parent record may be referenced by multiple claim or medicine rows. The ERD image is included as supplied.

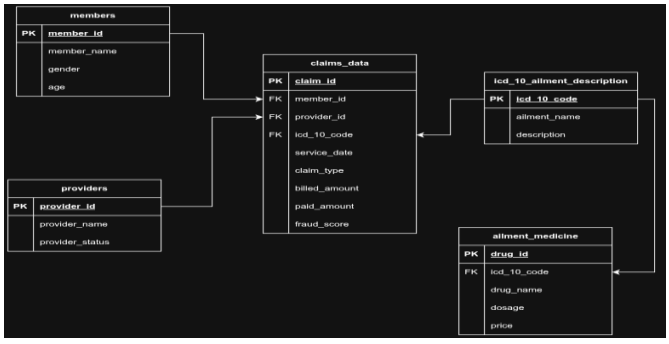


Fig. 1. ERD Diagram.

B. Integrity and implementation

Primary keys provide entity integrity and the foreign keys prevent claims or medicine rows from referencing missing parent records when constraints are enforced (referential integrity). Numeric types are used for identifiers, age, monetary values, and the score. Text types specifically VARCHAR are used for names, labels, descriptions, and dosage, DATE stores the service date. The complete CREATE TABLE statements and one INSERT example for each entity are also shown as SQL queries

The five supplied CSVs each contain ten data rows. The companion SQL script includes LOAD DATA LOCAL INFILE examples and is ordered so parent tables are loaded before dependent tables to help avoid conflicts

Below is a table showing the ERD's attributes

Table	Key and principal attributes
members	*member_id* PK, name, gender, age
providers	*provider_id* PK, name, status
ICD_10_ailment_description	*icd_10_code* PK, ailment name, description
claims_data	*claim_id* PK, member, provider ,code FKs, date, type, amounts, score
ailment_medicine	*drug_id* PK, code FK, drug, dosage, price

C. Example INSERT statements

The following statements add one row to each entity. The claims example depends on the example member and provider

```
INSERT INTO members (member_id, member_name, gender, age)

VALUES (9001, 'Example Member', 'Not stated', 45),

INSERT INTO providers (provider_id, provider_name, provider_status)

VALUES (9001, 'Example Health Centre', 'Active'),

INSERT INTO ICD_10_ailment_description (icd_10_code, ailment_name,
description)

VALUES ('Z00.00', 'General examination', 'Illustrative sample entry.'),

INSERT INTO claims_data (claim_id, members_id, provider_id, icd_10_code,

service_date, claim_type, billed_amount, paid_amount, fraud_score)

VALUES (9001, 9001, 9001, 'E11.9', '2025-11-01', 'Outpatient', 150.00, 120.00,
0.0500),

INSERT INTO ailment_medicine (drug_id, icd_10_code, drug_name, dosage, price)

VALUES (9001, 'E11.9', 'Example medicine', '1 tablet daily', 5.00),
```

III. NORMALISATION AND DESIGN JUSTIFICATION

Normalisation is assessed against the attributes and functional dependencies shown in the current schema. First normal form (1NF) requires each row to have a unique key and each field to hold a single value rather than a repeating list. The provided tables use one value per column in their sample records. The dosage and description fields are text values and they are not lists of multiple values in the supplied rows.

Second normal form (2NF) requires 1NF and no non-key attribute to depend on only part of a candidate key. Each table currently has a single-column primary key, so there is no proper subset of that key on which a partial dependency could exist. 2NF is therefore satisfied by the key structure

Third normal form (3NF) requires 2NF and no non-key attribute to depend transitively on a key through another non-key attribute. The dependencies represented are: *member_id* → member name, gender, age, *provider_id* → provider name, status, *icd_10_code* → ailment name, description, *claim_id* → member/provider/code references, service date, claim type, amounts, score, and *drug_id* → code, drug name, dosage, price. No additional non-key to non-key dependency is represented within any one table, so each table is in 3NF

Separating member, provider, and diagnosis details from claims avoids repeating those descriptions on every claim lets foreign keys maintain references. Similarly, diagnosis descriptions are stored once in the diagnosis table rather than repeated in every medicine or claim row

IV. SQL QUERIES AND RESULTS

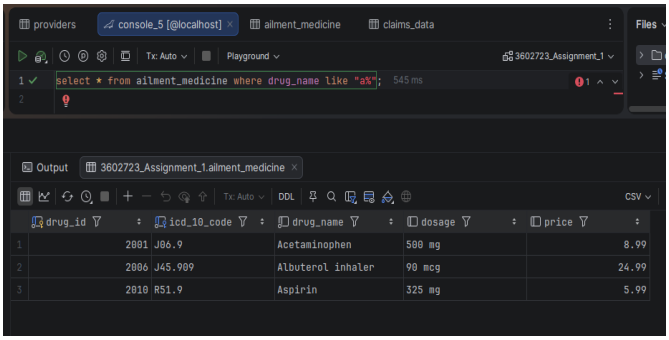
The following queries and result summaries reflect the supplied screenshots. Their outputs demonstrate filtering, a left join, a comparison, aggregation, and update statements

Query 1 - LIKE

```
SELECT * FROM ailment_medicine

WHERE drug_name LIKE 'a%',
```

The screenshot displays Acetaminophen, Albuterol inhaler, and Aspirin (3 rows).



Query 2 - LEFT JOIN

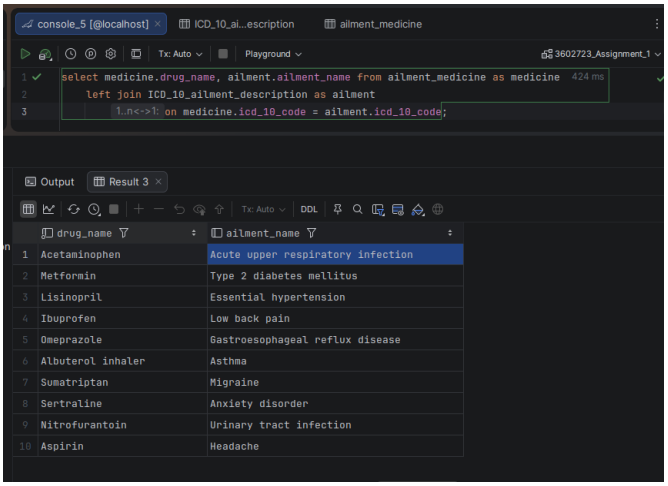
```
SELECT medicine.drug_name, ailment.ailment_name

FROM ailment_medicine AS medicine

LEFT JOIN ICD_10_ailment_description AS ailment

 ON medicine.icd_10_code = ailment.icd_10_code,
```

The screenshot shows ten medicine/ailment pairs



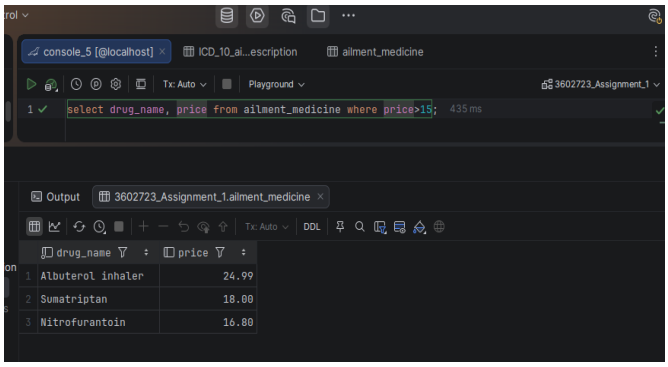
Query 3 - Greater than(>)

```
SELECT drug_name, price

FROM ailment_medicine

WHERE price > 15,
```

The result contains Albuterol inhaler (24.99), Sumatriptan (18.00), and Nitrofurantoin (16.80).



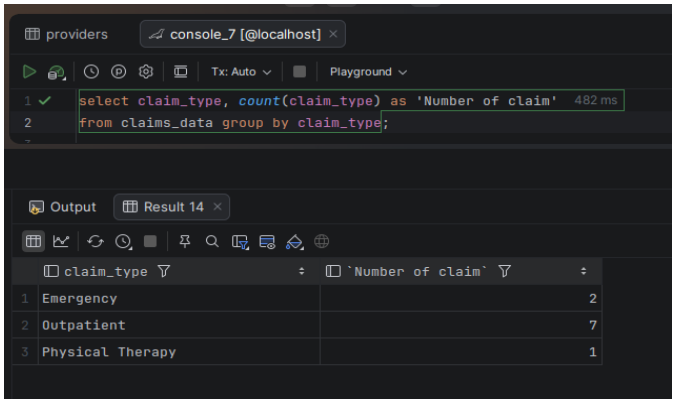
Query 4 - GROUP BY

```
SELECT claim_type, COUNT(claim_type) AS `Number of claim`

FROM claims_data

GROUP BY claim_type,
```

The result counts are Emergency: 2, Outpatient: 7, and Physical Therapy: 1.

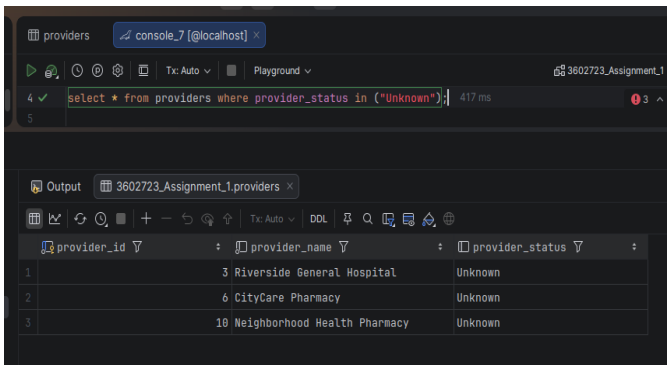


Query 5 - IN

```
SELECT * FROM providers

WHERE provider_status IN ('Unknown'),
```

The screenshot shows provider IDs 3, 6, and 10 with status Unknown.



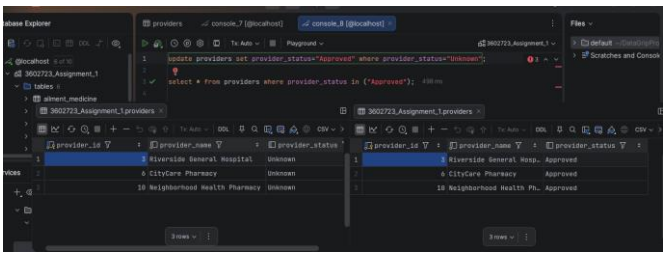
Query 6 - UPDATE

```
UPDATE providers

SET provider_status = 'Approved'

WHERE provider_status = 'Unknown',
```

The screenshot shows three provider rows changed to Approved.



V. DISCUSSION OF COMPLEXITY

A more complex feature for this scenario would be producing reliable fraud-model features from a member's claim history. A single claim's amount or diagnosis can be selected with ordinary SQL, but useful patterns may depend on other things like time windows, repeat visits, changes in claim frequency, and combinations of diagnosis and payment behaviour and so much more

Other difficulties encountered in SQL are a few errors and advanced SQL queries that required diving into rabbit holes. Such errors and complex queries include:

```
ERROR 4166 (HY000): The used command is not allowed because the MariaDB
server or client has disabled the local infile capability

MariaDB [3602723_Assignment_1]> SET GLOBAL local_infile = 1;

MariaDB [3602723_Assignment_1]> LOAD DATA LOCAL INFILE
'Desktop/Uni/CSCU9B3 Databases/CSCU9B3-Assignment-1/CSVs 2
import/ailment_medicine.csv' INTO TABLE ailment_medicine FIELDS
TERMINATED BY ',' IGNORE 1 LINES;

SOLUTION: mysql --local-infile=1 -u root -p
```

The difficulties above were from trying to import CSV files but the MySQL server was having none of it with regular LOAD DATA

VI. CONCLUSION

The above design and implementation is useful for storing and querying a small claims dataset. Its separate entities and foreign keys supports basic referential and entity integrity, and the existing query screenshots illustrate the executed SQL queries. The table level dependencies support 1NF, 2NF, and 3NF under the stated assumptions

REFERENCES

[1] “SQL Tutorial,” W3Schools, [Online Tutorials and courses]. Available at: https://www.w3schools.com/sql/